

SQL Queries

Step 1: Create the Database

```
CREATE DATABASE IMDB;  
USE IMDB;
```

Step 2: Define Tables with Relationships

1. Movies Table

```
CREATE TABLE Movies (  
    MovieID INT AUTO_INCREMENT PRIMARY KEY,  
    Title VARCHAR(255) NOT NULL,  
    ReleaseYear INT  
);
```

2. Media Table (Video/Image)

```
CREATE TABLE Media (  
    MediaID INT AUTO_INCREMENT PRIMARY KEY,  
    MovieID INT,  
    MediaType ENUM('Video', 'Image') NOT NULL,  
    URL VARCHAR(255),  
    FOREIGN KEY (MovieID) REFERENCES Movies(MovieID)  
);
```

3. Genres Table

```
CREATE TABLE Genres (  
    GenreID INT AUTO_INCREMENT PRIMARY KEY,  
    GenreName VARCHAR(50) NOT NULL  
);
```

4. MovieGenres Table (Many-to-Many Relationship)

```
CREATE TABLE MovieGenres (  
    MovieID INT,  
    GenreID INT,  
    PRIMARY KEY (MovieID, GenreID),
```

```
        FOREIGN KEY (MovieID) REFERENCES Movies(MovieID),  
        FOREIGN KEY (GenreID) REFERENCES Genres(GenreID)  
    );
```

5. Users Table

```
CREATE TABLE Users (  
    UserID INT AUTO_INCREMENT PRIMARY KEY,  
    Username VARCHAR(255) NOT NULL  
);
```

6. Reviews Table

```
CREATE TABLE Reviews (  
    ReviewID INT AUTO_INCREMENT PRIMARY KEY,  
    MovieID INT,  
    UserID INT,  
    ReviewText TEXT,  
    Rating INT,  
    FOREIGN KEY (MovieID) REFERENCES Movies(MovieID),  
    FOREIGN KEY (UserID) REFERENCES Users(UserID)  
);
```

7. Artists Table

```
CREATE TABLE Artists (  
    ArtistID INT AUTO_INCREMENT PRIMARY KEY,  
    Name VARCHAR(255) NOT NULL  
);
```

8. Skills Table

```
CREATE TABLE Skills (  
    SkillID INT AUTO_INCREMENT PRIMARY KEY,  
    SkillName VARCHAR(50) NOT NULL  
);
```

9. ArtistSkills Table (Many-to-Many Relationship)

```
CREATE TABLE ArtistSkills (  
    ArtistID INT,  
    SkillID INT,  
    PRIMARY KEY (ArtistID, SkillID),  
    FOREIGN KEY (ArtistID) REFERENCES Artists(ArtistID),
```

```
        FOREIGN KEY (SkillID) REFERENCES Skills(SkillID)
    );
```

10. MovieRoles Table (Artists with Multiple Roles in a Movie)

```
CREATE TABLE MovieRoles (
    RoleID INT AUTO_INCREMENT PRIMARY KEY,
    MovieID INT,
    ArtistID INT,
    RoleDescription VARCHAR(255),
    FOREIGN KEY (MovieID) REFERENCES Movies(MovieID),
    FOREIGN KEY (ArtistID) REFERENCES Artists(ArtistID)
);
```

Output:

1. List All Movies with Their Media (Videos/Images)

```
mysql> SELECT
->     m.Title AS MovieTitle,
->     med.MediaType,
->     med.URL
-> FROM
->     Movies m
-> JOIN
->     Media med ON m.MovieID = med.MovieID;
```

MovieTitle	MediaType	URL
Inception	Video	inception_trailer.mp4
K.G.F: Chapter 2	Video	kgf2_trailer.mp4
K.G.F: Chapter 2	Image	kgf2_poster.jpg

3 rows in set (0.02 sec)

```
mysql>
```

2. Find All Genres for a Specific Movie

```
mysql> SELECT
->     m.Title AS MovieTitle,
->     g.GenreName
-> FROM
->     Movies m
-> JOIN
->     MovieGenres mg ON m.MovieID = mg.MovieID
-> JOIN
->     Genres g ON mg.GenreID = g.GenreID
-> WHERE
->     m.movieID = 2;
```

MovieTitle	GenreName
K.G.F: Chapter 2	Action
K.G.F: Chapter 2	Drama
K.G.F: Chapter 2	Thriller
K.G.F: Chapter 2	Crime
K.G.F: Chapter 2	Gangster

5 rows in set (0.00 sec)

3. Find All Reviews for a Specific Movie

```
mysql> SELECT
->     m.Title AS MovieTitle,
->     u.UserName,
->     r.ReviewText,
->     r.Rating
-> FROM
->     Reviews r
-> JOIN
->     Movies m ON r.MovieID = m.MovieID
-> JOIN
->     Users u ON r.UserID = u.UserID
-> WHERE
->     m.MovieID = 2;
```

MovieTitle	UserName	ReviewText	Rating
K.G.F: Chapter 2	vidhyaprakash	Great movie!	5
K.G.F: Chapter 2	kundhan	Mind-blowing movie!	9

2 rows in set (0.00 sec)

4. List All Artists with Their Skills

```
mysql> SELECT
->     a.Name AS ArtistName,
->     s.SkillName
-> FROM
->     Artists a
-> JOIN
->     ArtistSkills aks ON a.ArtistID = aks.ArtistID
-> JOIN
->     Skills s ON aks.SkillID = s.SkillID;
```

ArtistName	SkillName
Prashanth Neel	Directing
Prashanth Neel	Screenplay
Prashanth Neel	Story
Yash as Rocky	Acting
Vijay	Acting

5 rows in set (0.00 sec)

5. List All Movies and Roles Played by Artists

```
mysql> SELECT
->     m.Title AS MovieTitle,
->     a.Name AS ArtistName,
->     mr.RoleDescription
-> FROM
->     MovieRoles mr
-> JOIN
->     Movies m ON mr.MovieID = m.MovieID
-> JOIN
->     Artists a ON mr.ArtistID = a.ArtistID;
+-----+-----+-----+
| MovieTitle | ArtistName | RoleDescription |
+-----+-----+-----+
| K.G.F: Chapter 2 | Prashanth Neel | Director, Writer, Screenplay, Dialogues |
| K.G.F: Chapter 2 | Yash as Rocky | Protagonist |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

6. Find All Movies by Genre

```
mysql> SELECT
->     m.Title AS MovieTitle,
->     g.GenreName
-> FROM
->     MovieGenres mg
-> JOIN
->     Movies m ON mg.MovieID = m.MovieID
-> JOIN
->     Genres g ON mg.GenreID = g.GenreID
-> WHERE
->     g.GenreName = 'Action';
+-----+-----+
| MovieTitle | GenreName |
+-----+-----+
| K.G.F: Chapter 2 | Action |
| Master | Action |
+-----+-----+
2 rows in set (0.00 sec)
```

7. Find Average Rating for Each Movie

```
mysql> SELECT
->     m.Title AS MovieTitle,
->     AVG(r.Rating) AS AverageRating
-> FROM
->     Reviews r
-> JOIN
->     Movies m ON r.MovieID = m.MovieID
-> GROUP BY
->     m.MovieID;
```

MovieTitle	AverageRating
K.G.F: Chapter 2	7.0000
Inception	4.0000

2 rows in set (0.02 sec)

8. Find All Users Who Reviewed a Specific Movie

```
mysql> SELECT
->     u.UserName,
->     r.ReviewText,
->     r.Rating
-> FROM
->     Users u
-> JOIN
->     Reviews r ON u.UserID = r.UserID
-> WHERE
->     r.MovieID = 2;
```

UserName	ReviewText	Rating
vidhyaprakash	Great movie!	5
kundhan	Mind-blowing movie!	9

2 rows in set (0.00 sec)

9. List All Skills of Artists Who Worked in a Specific Movie

```
mysql> SELECT
->     a.Name AS ArtistName,
->     s.SkillName
-> FROM
->     Artists a
-> JOIN
->     ArtistSkills aks ON a.ArtistID = aks.ArtistID
-> JOIN
->     Skills s ON aks.SkillID = s.SkillID
-> JOIN
->     MovieRoles mr ON a.ArtistID = mr.ArtistID
-> WHERE
->     mr.MovieID = 2;
```

ArtistName	SkillName
Prashanth Neel	Directing
Prashanth Neel	Screenplay
Prashanth Neel	Story
Yash as Rocky	Acting

4 rows in set (0.01 sec)

10. Find All Movies with Their Genres and Average Ratings

```
mysql> SELECT
->     m.Title AS MovieTitle,
->     GROUP_CONCAT(DISTINCT g.GenreName) AS Genres,
->     AVG(r.Rating) AS AverageRating
-> FROM
->     Movies m
-> LEFT JOIN
->     MovieGenres mg ON m.MovieID = mg.MovieID
-> LEFT JOIN
->     Genres g ON mg.GenreID = g.GenreID
-> LEFT JOIN
->     Reviews r ON m.MovieID = r.MovieID
-> GROUP BY
->     m.MovieID;
```

MovieTitle	Genres	AverageRating
Inception	Science Fiction	4.0000
K.G.F: Chapter 2	Action,Crime,Drama,Gangster,Thriller	7.0000
Master	Action	NULL

3 rows in set (0.02 sec)

11. Find All Movies Where a Specific Artist Has Played a Role

```
mysql> SELECT
->     m.Title AS MovieTitle,
->     mr.RoleDescription
-> FROM
->     Movies m
-> JOIN
->     MovieRoles mr ON m.MovieID = mr.MovieID
-> JOIN
->     Artists a ON mr.ArtistID = a.ArtistID
-> WHERE
->     a.Name = 'Prashanth Neel';
```

MovieTitle	RoleDescription
K.G.F: Chapter 2	Director, Writer, Screenplay, Dialogues

```
1 row in set (0.00 sec)
```

12. Find All Artists Who Share a Specific Skill

```
mysql> SELECT
->     a.Name AS ArtistName,
->     s.SkillName
-> FROM
->     Artists a
-> JOIN
->     ArtistSkills aks ON a.ArtistID = aks.ArtistID
-> JOIN
->     Skills s ON aks.SkillID = s.SkillID
-> WHERE
->     s.SkillName = 'Acting';
```

ArtistName	SkillName
Yash as Rocky	Acting
Vijay	Acting

```
2 rows in set (0.00 sec)
```